

DPE Lakehouse

Tutoriel d'exploitation

Comment lancer chaque composant, pourquoi il est là, et ce qu'il faut savoir en dire.

Source	ADEME, base des DPE logements existants
Volumes	15,3 M de lignes ingérées, 7 849 191 en entrepôt
Stack	Python, Spark/Scala, PostgreSQL, dbt, Airflow, Docker, Power BI
Dépôt	github.com/ebenezer-ngblogni/dpe-lakehouse

Table des matières

1 Pourquoi ce projet	3
1.1 Le problème choisi	3
1.2 Ce que le projet démontre	3
1.3 L'architecture en une phrase	3
2 Démarrer la pile	4
2.1 Prérequis	4
2.2 Lancer les services	4
3 Étape 1 — Ingestion (Python)	5
3.1 Lancer	5
3.2 Pourquoi Python ici, et pas Spark	5
3.3 Les deux idées à retenir	5
3.3.1 Pagination par curseur	5
3.3.2 Promotion atomique	5
3.3.3 Le choix du format de transport	5
4 Étape 2 — Nettoyage (Spark / Scala)	7
4.1 Lancer	7
4.2 Pourquoi Scala et pas PySpark	7
4.3 Les trois transformations	7
4.3.1 Déduplication par fenêtre	7
4.3.2 Règles qualité avec motif conservé	7
4.3.3 Le seuil de géocodage, corrigé après mesure	8
5 Étape 3 — Entrepôt (PostgreSQL)	9
5.1 Lancer	9
5.2 Pourquoi PostgreSQL et pas Snowflake ou BigQuery	9
5.3 Deux optimisations concrètes	9
6 Étape 4 — Modélisation (dbt)	10
6.1 Lancer	10
6.2 L'interface : le graphe de lignage	10
6.3 Pourquoi dbt	10
6.4 Le piège de la moyenne de taux	10
6.5 Les deux tests rouges, assumés	11
7 Étape 5 — Orchestration (Airflow)	12
7.1 Lancer	12
7.2 L'interface : ce qu'il faut savoir montrer	12
7.3 Le DAG, en six lignes	12
7.4 La fenêtre glissante	12
8 Étape 6 — Restitution (Power BI)	13
8.1 Lancer	13
8.2 Les deux pièges Power BI, silencieux tous les deux	13
8.3 Pourquoi la carte est départementale	13

9 Les résultats à connaître	14
9.1 Les quatre chiffres qui portent le projet	14
9.2 Le résultat le plus fort	14
9.3 Les limites, à énoncer soi-même	14
10 Questions probables en entretien	15
10.1 « Pourquoi Spark pour 15 millions de lignes ? Pandas suffirait. »	15
10.2 « Comment garantisiez-vous l'idempotence ? »	15
10.3 « Qu'est-ce qui vous a le plus surpris ? »	15
10.4 « Que feriez-vous différemment ? »	15
10.5 « Montrez-moi une décision technique que vous regrettez. »	15

1. Pourquoi ce projet

1.1 Le problème choisi

La France compte environ 5,8 millions de logements classés F ou G, les **passoires thermiques**. La loi Climat et Résilience les interdit progressivement à la location. Savoir *où* elles se trouvent et *à quoi* elles ressemblent est une question publique, et l'ADEME publie la matière première en accès libre.

Ce que je répons si on me demande « pourquoi ce sujet ? »

Trois raisons, dans cet ordre :

1. **Le volume est réel.** 15,3 millions de lignes obligent à traiter en distribué. Sur 50 000 lignes, Spark serait une pose ; ici il est nécessaire.
2. **La donnée est sale, et de façon intéressante.** Codes postaux à la place des codes INSEE, communes fusionnées préfixées oLd, noms de colonnes corrompus par l'export CSV. Ce sont des problèmes qu'on ne rencontre pas dans un jeu de données pédagogique.
3. **Le résultat se raconte.** La part de passoires s'effondre après la réglementation thermique de 1974. C'est un fait mesuré sur des données de terrain, pas une illustration.

1.2 Ce que le projet démontre

Compétence	Où elle se voit
Ingestion résiliente	Pagination par curseur, partitions mensuelles, reprise sans doublon
Traitement distribué	Spark 3.5 en Scala, fenêtres, jointures, 15,3 M de lignes
Qualité de données	720 483 rejets tracés avec leur motif, jamais silencieux
Modélisation	Schéma en étoile dbt, agrégats commune et département
Orchestration	DAG Airflow à six tâches, chacune reprenable
Restitution	Rapport Power BI, carte de France à 96 points lisibles
Rigueur	74 tests automatisés, intégration continue GitHub Actions

1.3 L'architecture en une phrase

Une **architecture médaille** : la donnée brute atterrit en *bronze* sans transformation, est nettoyée en *silver*, puis modélisée en *gold* (les marts) pour la restitution.

Pourquoi trois couches et pas une seule

Parce que chaque couche répond à une question différente.

Bronze répond à « qu'a dit la source, exactement ? ». On n'y touche à rien : si une règle métier change, on rejoue depuis le bronze sans retélécharger 15 millions de lignes — l'API met environ cinq heures.

Silver répond à « quelles lignes sont exploitables ? ». C'est là que vivent la déduplication et les règles qualité.

Gold répond à « que veut voir l'analyste ? ». Les agrégats y sont pré-calculés pour que Power BI réponde instantanément.

2. Démarrer la pile

2.1 Prérequis

```
# Depuis la racine du dépôt
cd ~/Bureau/lab/DataEng

# Le lac vit sur une partition séparée (1 To). Si `data` est un lien mort :
udisksctl mount -b /dev/sda5
ls data/          # doit lister bronze/ silver/ exports/

# Environnement Python
make setup        # crée .venv et installe les dépendances
```

Le piège des identifiants Docker

Les conteneurs écrivent dans des volumes montés depuis l'hôte. Si l'UID du conteneur ne correspond pas au vôtre, l'écriture échoue avec un `Permission denied` peu bavard. Il faut donc exporter ces variables *avant* de lancer la pile :

```
export AIRFLOW_UID=$(id -u)
export HOST_UID=$(id -u)
export DOCKER_GID=$(getent group docker | cut -d: -f3)
```

DOCKER_GID est nécessaire parce qu'Airflow soumet les jobs Spark via `docker exec` : il lui faut le droit de parler au démon Docker.

2.2 Lancer les services

```
make up            # équivalent de : docker compose up -d
docker compose ps  # vérifier que tout est "healthy"
```

Service	Adresse	Rôle
Airflow	http://localhost:8080	Orchestration (admin / admin)
MinIO	http://localhost:9003	Stockage objet compatible S3
PostgreSQL	localhost:5434	Entrepôt analytique
dbt docs	http://localhost:8081	Documentation et lignage

Pourquoi ces ports décalés

5434 et non 5432, 9002/9003 et non 9000/9001 : les ports par défaut étaient déjà pris par un PostgreSQL local et un autre projet. Le décalage est volontaire et propagé partout — `docker-compose.yml`, `profiles.yml`, le `Makefile`, le code Scala. C'est typiquement le genre de détail qu'un recruteur apprécie de vous entendre expliquer.

3. Étape 1 — Ingestion (Python)

3.1 Lancer

```
make ingest-sample # un seul mois, environ 2 minutes – pour tester
make ingest-full   # tout l'historique depuis juillet 2021, environ 5 h
make ingest-status # compare l'état local à la source, partition par partition
```

3.2 Pourquoi Python ici, et pas Spark

L'ingestion est un travail **d'entrées-sorties réseau**, pas de calcul. On attend l'API, on écrit sur disque, on recommence. Spark n'apporterait rien : son intérêt est de paralléliser du calcul sur des données déjà présentes. Python avec PyArrow suffit et se déploie en trois lignes.

3.3 Les deux idées à retenir

Pagination par curseur

L'API ADEME (Data Fair) plafonne à 10 000 lignes par requête et fournit le lien suivant dans un en-tête HTTP Link. On ne calcule donc jamais d'*offset* soi-même : on suit le curseur.

```
# Le lien `next` embarque déjà tous les paramètres de la requête.
next_link = response.links.get("next")
url = next_link.get("url") if next_link else None
```

Pourquoi c'est mieux qu'un offset

Avec OFFSET *n*, si une ligne est insérée en cours de parcours, tout décale et on saute ou duplique des enregistrements. Le curseur est stable : il pointe une position logique dans le jeu, pas un rang.

Promotion atomique

Chaque mois est écrit dans un répertoire de *staging*, puis promu d'un seul coup, accompagné d'un manifeste :

```
def is_partition_complete(self, table, year, month) -> bool:
    """Une partition est réputée chargée si son manifeste est présent
    ET complet."""
    manifest = self.read_manifest(table, year, month)
    return manifest is not None and manifest.is_complete
```

Le problème que ça résout

Si le processus meurt à 80 % d'un mois, une écriture directe laisserait une partition à moitié remplie — indiscernable d'une partition complète. À la reprise, le pipeline la croirait faite et perdrait 20 % des données, silencieusement.

Avec la promotion atomique, une partition incomplète n'a pas de manifeste : elle est simplement rejouée.

C'est ce qui rend l'ingestion idempotente : la relancer dix fois donne le même résultat qu'une fois.

Le choix du format de transport

Une décision prise après mesure, pas par principe

L'API sert du JSON ou du CSV. J'ai mesuré les deux : le CSV divise **par 2,8** le temps de chargement, parce qu'il évite de construire un objet Python par ligne.

Le prix à payer : l'export CSV de l'API corrompt certains noms de colonnes en remplaçant un underscore par une espace — le schéma déclare `conso_5_usages_par_m2_ef`, le CSV émet `conso_5 usages_par_m2_ef`. On répare à la lecture, et trois tests de non-régression verrouillent la correction.

4. Étape 2 — Nettoyage (Spark / Scala)

4.1 Lancer

```
make build      # compile le jar avec sbt-assembly
make test-scala # 15 tests unitaires
make silver     # bronze -> silver, sur la totalité du lac
```

4.2 Pourquoi Scala et pas PySpark

Trois arguments, du plus fort au plus faible :

1. **Scala est la langue native de Spark.** Le moteur est écrit en Scala ; on parle directement à l'API, sans passerelle Py4J.
2. **Le typage attrape les erreurs à la compilation.** Une faute de frappe sur un nom de colonne ou un mauvais type est signalée avant de lancer un job qui tourne vingt minutes.
3. **C'est ce que demandent les offres.** Beaucoup d'équipes data historiques sont en Scala ; savoir y écrire ouvre des portes que PySpark seul n'ouvre pas.

L'argument que je n'utilise *pas* : « c'est plus rapide ». Depuis les DataFrames, PySpark et Scala passent par le même optimiseur Catalyst et les performances sont comparables. Le dire serait faux, et un bon recruteur le sait.

4.3 Les trois transformations

Déduplication par fenêtre

```
val fenetre = Window
  .partitionBy(col("numero_dpe"))
  .orderBy(
    col("date_derniere_modification_dpe").desc_nulls_last,
    col("date_reception_dpe").desc_nulls_last
  )

df.withColumn("_rang", row_number().over(fenetre))
  .filter(col("_rang") === 1)
  .drop("_rang")
```

Résultat mesuré : 0 ligne supprimée

J'attendais des milliers de doublons. Il n'y en a aucun — le numéro de DPE est réellement unique dans le lot ingéré. **J'ai gardé le code et documenté le résultat** plutôt que de le retirer : c'est un garde-fou qui protège une hypothèse, et savoir qu'elle tient a de la valeur.

Si on me demande « pourquoi garder du code qui ne fait rien ? », la réponse est : il ne fait rien *aujourd'hui*. Le jour où l'ADEME republie un diagnostic, il fera son travail.

Règles qualité avec motif conservé

```
private def motifRejet: Column =
  when(col("numero_dpe").isNull, "identifiant_absent")
  .when(!col("etiquette_dpe").isin(EtiquettesValides: _*), "etiquette_dpe_invalide")
  .when(col("date_etablissement_dpe").isNull, "date_etablissement_absente")
  .when(col("code_insee_ban").isNull, "commune_absente")
  .when(col("surface_habitable_logement") <= 0, "surface_invalide")
  // Un logement de plus de 1 000 m2 relève de la saisie erronée.
```



```
.when(col("surface_habitable_logement") > 1000, "surface_aberrante")  
.when(col("conso_5_usages_par_m2_ep") < 0, "consommation_invalide")
```

Pourquoi écrire les rejets au lieu de les jeter

720 483 lignes sont écartées. Si on se contentait d'un `filter`, elles disparaîtraient sans trace et personne ne saurait pourquoi le total ne tombe pas juste.

Ici chaque rejet garde son motif et part dans `silver/dpe_rejets`. On peut donc répondre à « combien de lignes perd-on, et pour quelle raison ? » — question que pose systématiquement un responsable data.

Le seuil de géocodage, corrigé après mesure

```
val SeuilScoreBan: Double = 0.64 // mediane mesuree (etait 0.8 : 79 % marques)
```

J'avais fixé le seuil à 0,8 « parce que ça semblait raisonnable ». Résultat : 79 % des lignes signalées comme douteuses — un filtre qui marque quatre lignes sur cinq ne filtre rien. J'ai regardé la distribution réelle, pris la médiane, et surtout basculé sur le *statut* de géocodage fourni par la source, qui est un verdict et non un score.

5. Étape 3 — Entrepôt (PostgreSQL)

5.1 Lancer

```
make warehouse # silver -> PostgreSQL, fenêtre depuis 2024
```

```
# Se connecter pour vérifier
```

```
docker exec -it dpe_warehouse psql -U dpe -d dpe
```

```
-- Quelques requêtes de contrôle
```

```
SELECT count(*) FROM silver.dpe_courant; -- 7 849 191
```

```
SELECT count(*) FROM silver.dpe_rejets; -- 720 483
```

```
\d marts.*
```

5.2 Pourquoi PostgreSQL et pas Snowflake ou BigQuery

Parce que le volume ne le justifie pas. 7,8 millions de lignes sur 40 colonnes tiennent dans 2,7 Go. Un PostgreSQL sur un poste répond en quelques secondes. Payer un entrepôt cloud pour ça serait une dépense sans contrepartie.

Ce que je dis ensuite, parce que c'est la vraie question : **à partir de quand je changerais**. Trois signaux — quand plusieurs équipes requêtent en concurrence, quand les volumes dépassent ce qu'une machine tient en mémoire, ou quand il faut séparer stockage et calcul pour maîtriser les coûts. Aucun n'est atteint ici.

5.3 Deux optimisations concrètes

- **Projection de colonnes** : 40 colonnes chargées sur 78 disponibles. L'entrepôt passe de 5 825 Mo à 2 700 Mo. On ne charge que ce que les marts consomment.
- **Fenêtre temporelle** : seules les données depuis 2024 partent en entrepôt. Le lac, lui, garde tout l'historique. C'est un arbitrage coût de stockage contre profondeur d'analyse, assumé et documenté.

6. Étape 4 — Modélisation (dbt)

6.1 Lancer

```
cd dbt/dpe_analytics

dbt deps                # installe dbt_utils
dbt seed --profiles-dir . # charge le référentiel des départements
dbt run --profiles-dir .  # construit les modèles
dbt test --profiles-dir . # 44 tests de contrat
dbt docs generate --profiles-dir .
dbt docs serve --profiles-dir . --port 8081
```

6.2 L'interface : le graphe de lignage

dbt docs serve ouvre une interface web sur <http://localhost:8081>. Deux choses à savoir montrer :

1. **Le graphe de lignage** (bouton en bas à droite). Il affiche automatiquement les dépendances entre modèles : stg_dpe → fct_dpe et dim_commune → les trois marts. **Ce graphe n'est pas dessiné à la main** : dbt le déduit des ref() dans le SQL.
2. **La fiche d'un modèle** : sa description, ses colonnes, ses tests, et le SQL compilé. C'est la documentation qui ne peut pas mentir, puisqu'elle est générée depuis le code.

6.3 Pourquoi dbt

Ce que dbt apporte qu'un script SQL n'apporte pas

- **Le lignage gratuit.** On écrit {{ ref('fct_dpe') }} au lieu du nom de table ; dbt en déduit l'ordre d'exécution et le graphe de dépendances.
- **Les tests comme contrat.** Unicité, non-nullité, plages de valeurs, cohérence arithmétique — déclarés en YAML à côté du modèle, pas dans un script séparé qu'on oublie de lancer.
- **La documentation versionnée.** Elle vit dans le dépôt et suit le code.

6.4 Le piège de la moyenne de taux

```
-- CORRECT : rapport de sommes
round(100.0 * sum(nb_passoires) / nullif(sum(nb_dpe), 0), 2) AS taux_passoires_pct

-- FAUX : moyenne de taux, dès que les dénominateurs diffèrent
avg(taux_passoires_pct)
```

À savoir expliquer par cœur

Une commune de 30 diagnostics et une préfecture de 40 000 ne pèsent pas pareil. Faire la moyenne de leurs taux donnerait autant de poids à l'une qu'à l'autre.

C'est pour cette raison que **les taux sont des colonnes pré-calculées en SQL** et non des mesures Power BI : le modèle sémantique est en lecture seule, donc je ne peux pas y écrire de DAX. J'ai fait remonter le calcul dans dbt, où il est juste et testé.

6.5 Les deux tests rouges, assumés

Sur 44 tests dbt, 42 passent et 2 échouent. **Je les laisse en rouge.**

Ils signalent un vrai doublon dans la source : l'ADEME a publié deux fois le même diagnostic, une version incomplète puis une version complète. Le test dit la vérité; l'assouplir pour obtenir un tableau vert reviendrait à masquer un défaut réel.

Si on me demande « pourquoi ne pas corriger ? » : parce que je n'ai pas encore tranché la cause racine, et qu'un correctif posé sans comprendre en créerait un autre. En attendant, le test documente le problème.

7. Étape 5 — Orchestration (Airflow)

7.1 Lancer

```
make up # démarre webserver + scheduler
# Interface : http://localhost:8080 (admin / admin)
```

7.2 L'interface : ce qu'il faut savoir montrer

Vue	Ce qu'elle apprend
Grid	Historique des exécutions. Chaque carré est une tâche : vert réussi, rouge échoué. On lit d'un coup d'œil quelle étape casse et depuis quand.
Graph	Le DAG lui-même, avec les dépendances. C'est la vue à ouvrir en entretien.
Logs	Sortie de chaque tâche. C'est là qu'on débogue.
Code	Le fichier Python du DAG, tel que le scheduler le lit.

7.3 Le DAG, en six lignes

```
debut >> ingestion >> transformation >> chargement \
>> dbt_build >> dbt_test >> dbt_docs >> fin
```

Chaque tâche est un `BashOperator` qui soumet un travail dans le conteneur adéquat :

```
transformation = BashOperator(
    task_id="spark_bronze_vers_silver",
    bash_command=(
        "docker exec dpe_spark spark-submit "
        "--class fr.dpelab.silver.DpeSilverJob --master 'local[*]' "
        f"--conf 'spark.local.dir={SPARK_DATA_ROOT}/tmp-spark' "
        f"{SPARK_JAR} --bronze-path {SPARK_BRONZE}/dpe_existant ..."
    ),
)
```

Pourquoi Airflow, et pas un cron

Un cron sait lancer une commande à une heure. Il ne sait pas :

- **gérer les dépendances** — ne pas charger l'entrepôt si le nettoyage a échoué ;
- **rejouer une seule étape** sans refaire les précédentes ;
- **montrer l'historique** : quelle exécution a échoué, quand, avec quels logs.

C'est exactement ce qu'apporte Airflow, et c'est pourquoi il est devenu le standard de fait.

Pourquoi `docker exec` plutôt qu'un opérateur Spark

Airflow tourne dans son conteneur, Spark dans le sien. Plutôt que d'installer Spark dans l'image Airflow — ce qui la ferait grossir et mêlerait deux responsabilités — Airflow *soumet* le travail au conteneur Spark. Chaque image reste spécialisée. En production on utiliserait le `SparkSubmitOperator` ou un cluster Kubernetes ; le principe est le même.

7.4 La fenêtre glissante

Le DAG ne rejoue pas les 15 millions de lignes chaque nuit : il traite une fenêtre glissante de trois mois. L'ADEME corrigeant parfois des diagnostics publiés, on repasse sur le passé récent sans tout refaire.

8. Étape 6 — Restitution (Power BI)

8.1 Lancer

```
make powerbi-export # exporte les marts en CSV

# ou de façon ciblée
python scripts/export_powerbi.py \
  --table mart_performance_departement --destination powerbi/data

git add powerbi/data && git commit && git push
```

Le rapport lit GitHub, pas votre machine

Faute de licence OneDrive pour téléverser un fichier, Power BI Service lit les CSV par leur **URL brute GitHub**. Conséquence directe : **l'actualisation dépend d'un git push**. C'est une limite du montage, pas un oubli — et il vaut mieux la dire soi-même que se la faire remarquer.

8.2 Les deux pièges Power BI, silencieux tous les deux

1. **Le délimiteur n'est pas détecté.** Tout atterrit dans une colonne unique. Il faut le forcer à « Virgule ».
2. **L'auto-typage en locale française casse les décimaux.** Le CSV écrit 8.69 avec un point, la locale attend une virgule, et Power Query bascule la colonne en texte — latitude et longitude comprises, ce qui vide la carte *sans message d'erreur*.

```
Table.TransformColumnTypes("#En-têtes promus",
  {{"code_departement", type text},
   {"nb_dpe", Int64.Type},
   {"taux_passoires_pct", type number},
   {"latitude", type number}, {"longitude", type number}}, "en-US")
```

Et le code département reste du texte

Typé en nombre, « 01 » perdrait son zéro initial et « 2A » ferait échouer la conversion. Le même piège existe côté dbt : le seed `ref_departement` déclare ses types explicitement.

8.3 Pourquoi la carte est départementale

La première version cartographiait les communes. À 23 628 communes géolocalisées, chaque point occupe moins d'un pixel : les cercles se recouvrent, plus rien n'est identifiable. J'ai essayé le rayon, l'opacité, la carte thermique. Aucun réglage ne corrige cela — **c'est le grain qui était en cause**, pas l'affichage. À 96 départements, chaque bulle redevient lisible. C'est la leçon que je retiens : quand un visuel ne passe pas, il faut souvent remonter au modèle, pas régler le visuel.

9. Les résultats à connaître

9.1 Les quatre chiffres qui portent le projet

Résultat	Ce qu'il dit
8,69 %	Part de passoires (F ou G) dans le parc diagnostiqué, soit 681 719 logements
17,5 → 0,0 %	Part de passoires selon la période de construction, d'avant 1948 à après 2013
Facteur 17	Entre la Creuse (34,6 %) et l'Hérault (2,1 %)
6 € contre 50 €	Coût annuel de chauffage au m ² , d'une étiquette A à une étiquette G

9.2 Le résultat le plus fort

La chute après **1974** — année de la première réglementation thermique, adoptée dans la foulée du choc pétrolier. Avant 1948, 17,5 % de passoires. Après 2013, 0,0 %, et **60,3 % des maisons neuves décrochent une étiquette A ou B.**

C'est l'effet d'une politique publique, mesuré sur des données de terrain.

9.3 Les limites, à énoncer soi-même

Biais de sélection

Un DPE se réalise à la vente ou à la location. Ces 7,8 millions de lignes décrivent le parc *qui change de mains*, pas le parc français. Les logements occupés durablement par leurs propriétaires sont sous-représentés — et ce sont souvent les plus anciens.

Une corrélation qu'il ne faut pas surinterpréter

À l'échelle du logement, le type de bâtiment pèse : 14,5 % de passoires pour les maisons contre 6,2 % pour les appartements. Mais à l'échelle départementale, le lien s'effondre — **r = 0,31**, soit 9 % de variance expliquée. L'agrégation dilue l'effet et le climat reprend le dessus.

Le titre du visuel le dit ainsi, plutôt que de laisser croire à un lien fort.

10. Questions probables en entretien

10.1 « Pourquoi Spark pour 15 millions de lignes ? Pandas suffirait. »

Non, et c'est mesurable : la déduplication brasse plus de 10 Go de *shuffle*. Sur un poste à 16 Go de RAM, Pandas sature. Spark décharge sur disque et termine. J'ai d'ailleurs dû rediriger `spark.local.dir` vers le disque du lac, parce que les fichiers temporaires remplissaient le disque système.

10.2 « Comment garantissez-vous l'idempotence ? »

Trois mécanismes, un par couche :

- **Ingestion** : promotion atomique par partition, avec manifeste de complétude.
- **Silver** : réécriture complète en `partitionOverwriteMode=dynamic` — seules les partitions produites sont remplacées.
- **Entrepôt** : `SaveMode.Overwrite` avec `truncate=true`, qui vide et recharge sans détruire les index.

Rejouer le DAG ne duplique rien.

10.3 « Qu'est-ce qui vous a le plus surpris ? »

Que **trois de mes hypothèses soient fausses**, et de façon mesurable : la déduplication ne supprime rien, les chaînes de remplacement représentent 0,01 % et non 8,9 %, et mon seuil de géocodage marquait 79 % des lignes.

Je les ai documentées dans le README plutôt que de les effacer. Un pipeline qui ne raconte que ses succès n'apprend rien à celui qui le reprend.

10.4 « Que feriez-vous différemment ? »

Trois choses :

1. **Intégrer l'export Power BI au DAG.** Il est aujourd'hui manuel parce qu'il dépend d'un `git push` ; il faudrait un jeton d'écriture dans l'orchestrateur.
2. **Trancher le doublon en silver.** Deux tests sont rouges et je sais pourquoi, mais pas encore d'où vient le doublon exactement.
3. **Passer les marts en incrémental.** Aujourd'hui ils sont reconstruits entièrement à chaque exécution ; à volume constant c'est acceptable, mais ça ne passera pas l'échelle.

10.5 « Montrez-moi une décision technique que vous regrettez. »

Avoir fixé le seuil de géocodage à 0,8 sans regarder la distribution. C'était une valeur *qui semblait raisonnable*, ce qui n'est pas un critère. La mesure a montré qu'elle marquait quatre lignes sur cinq. Depuis, je vérifie un seuil avant de l'écrire.